ARROWSTACK LEARN. BUILD. GROW.

1-Month Internship — Full Stack Development

A practical assignment handbook built around three progressive, portfolio-oriented industry projects.

Internship Domain	Full Stack Development	**Duration**	1 month / 4 weeks
Difficulty	Intermediate	**Projects**	Exactly 3
Who Can Participate	Students, recent graduates, career switchers and interns able to meet the stated prerequisites.	**Working Mode**	Individual, mentor-reviewed, evidence-based delivery.

Learning Objectives

- Apply full stack development knowledge to a realistic stakeholder problem rather than a tutorial-only exercise.
- Plan work, manage scope, document decisions and communicate progress professionally.
- Produce evidence of quality through testing, validation, review, measurement or structured feedback.
- Finish with artifacts that can be demonstrated in an interview, portfolio or mentor review.

Industry Relevance & Expected Outcome

The internship simulates the expectations of a junior professional working in full stack development. The intern is expected to move from requirements and planning to execution, validation and final handoff. The three projects intentionally increase in independence and portfolio value.

Recommended Prerequisites & Tools

- Basic programming/domain fundamentals and ability to work independently with documentation.
- Git, documentation tools and the project-specific development/analysis stack.

Month-at-a-Glance

Week 1	Research • requirements • scope • setup • design/method
Week 2	Core execution • development • analysis • creation
Week 3	Advanced work • testing • refinement • validation
Week 4	Finalization • documentation • presentation • submission

General Submission Rules

- Use only authorized/public/simulated data and systems. Never test real systems without explicit written authorization.
- Keep a dated work log and maintain organized project files with clear filenames.
- Cite external sources where research or public data is used; distinguish assumptions from observed evidence.
- Submit a concise README or equivalent handoff document so a mentor can reproduce or review the work.
- Final presentations should demonstrate the result, evidence, key decisions, limitations and next steps.

ARROWSTACK LEARN. BUILD. GROW.

Project 1: B2B Order Management Platform

Project Type: Foundation / Core Industry Task

1. Real-World Scenario

Develop a multi-role ordering system for a simulated distributor with products, carts, orders, invoices, stock status and admin controls.

2. Project Objective

Deliver a working, reviewable solution for the scenario using professional full stack development practices. The objective is not only to produce an output, but to show how you moved from problem definition to a defensible final result.

3. Business Use Case

A real organization could use this type of workflow to improve service quality, operational visibility, productivity, decision-making or customer/user experience. The intern should identify the primary stakeholder, the decision enabled by the output and the measurable benefit.

4. Problem Statement

Design and complete the full stack development project so that a stakeholder can reliably use or evaluate the final result. Define inputs, outputs, constraints, quality measures and the conditions under which the solution should be considered successful.

5. Required Features / Activities

- Role-based access
- REST API integration
- Relational database with migrations
- Validation and error handling
- Audit-friendly activity records
- Deployment-ready configuration

6. Functional / Professional Requirements

- Define the target user/stakeholder, scope boundaries and success metrics before execution.
- Use a repeatable workflow with traceable decisions and clearly organized artifacts.
- Handle realistic edge cases, invalid inputs, exceptions or uncertainty relevant to the domain.
- Include a validation/review step and retain evidence of the result.
- Document limitations and recommend realistic next improvements.

7. Technical / Method Requirements

- Recommended stack: React/Next.js, Node.js/NestJS, PostgreSQL, JWT, Docker
- Use current stable tooling that is accessible to the intern; reasonable alternatives are allowed when documented.
- Apply version control or a dated evidence folder appropriate to the domain.
- Follow responsible-use, security, privacy and data-integrity practices relevant to the project.

8. Suggested Technology Stack

React/Next.js, Node.js/NestJS, PostgreSQL, JWT, Docker

9. Step-by-Step Implementation Plan

1. Research the full stack development workflow and identify the target user, stakeholders, inputs, outputs and success metrics.
2. Convert the scenario into a written requirements document, acceptance criteria and delivery checklist.
3. Create the initial architecture/design/data model and establish a version-controlled workspace.
4. Implement the core workflow in small, testable increments and keep a decision log.
5. Validate the result using representative test cases or evidence and document known limitations.

10. Week-by-Week Plan

ARROWSTACK
LEARN. BUILD. GROW.

Week 1	Research, stakeholder/problem framing, requirements, dataset/assets, environment setup and initial design.
Week 2	Core implementation, analysis, content creation, research execution or prototype build.
Week 3	Advanced requirements, testing/validation, optimization, refinement and mentor review.
Week 4	Final QA/validation, documentation, presentation/demo, packaging and submission.

11. Deliverables

- Project source files or working analysis/design artifacts
- Requirements and implementation/research documentation
- Test/validation evidence
- README or user/technical guide
- Final presentation or demo
- Version-controlled project repository or organized submission folder

12. Expected Final Output

A complete, organized and presentation-ready project package that demonstrates the required workflow, includes evidence of quality/validation, and can be understood by a mentor without relying on undocumented steps.

13. Acceptance Criteria

- All stated core requirements are completed and mapped to evidence.
- The solution/workflow is reproducible by a mentor using the submitted instructions and files.
- Quality checks/tests/validation are documented and major known issues are disclosed.
- The final output is professionally organized, named consistently and ready for demonstration.
- The intern can explain key decisions, trade-offs, limitations and future improvements.

14. Evaluation Rubric — 100 Points

Criterion	Points
Requirements & domain understanding	15
Technical / professional execution	25
Quality, accuracy & validation	20
Problem-solving & independence	15
Documentation & reproducibility	10
Presentation & communication	10
Creativity / improvement thinking	5

15. Portfolio Value

Present the project on a resume as a measurable internship project; publish appropriate source/artifacts on GitHub or a portfolio; share a concise outcome-focused LinkedIn post; and use the project as an interview story covering problem, action, evidence and result.

16. Viva / Interview Questions

- What business or user problem does the project solve, and how did you define success?
- Which design, technical, analytical or research decision had the biggest impact on the result?
- How did you validate the quality or correctness of your work?
- What edge cases, limitations or risks did you identify?
- What would you change if this solution had to support 10× the users/data/workload?
- How would you explain the project to a non-technical stakeholder?
- Which part of the project required the most independent problem-solving?

17. Extension / Advanced Challenge

ARROWSTACK
LEARN. BUILD. GROW.

- Add monitoring/measurement or a stronger evaluation framework and compare results before vs. after.
- Introduce an additional user/stakeholder role or more realistic edge cases.
- Automate one repetitive step and document the productivity or quality benefit.
- Create a production/research-readiness roadmap covering security, scalability, governance or maintainability.

ARROWSTACK
LEARN. BUILD. GROW.

Project 2: Employee Service Request System

Project Type: Intermediate Real-World Task

1. Real-World Scenario

Build an internal portal for employees to raise IT/HR/facility requests, route them, track SLAs and view dashboards.

2. Project Objective

Deliver a working, reviewable solution for the scenario using professional full stack development practices. The objective is not only to produce an output, but to show how you moved from problem definition to a defensible final result.

3. Business Use Case

A real organization could use this type of workflow to improve service quality, operational visibility, productivity, decision-making or customer/user experience. The intern should identify the primary stakeholder, the decision enabled by the output and the measurable benefit.

4. Problem Statement

Design and complete the full stack development project so that a stakeholder can reliably use or evaluate the final result. Define inputs, outputs, constraints, quality measures and the conditions under which the solution should be considered successful.

5. Required Features / Activities

- Role-based access
- REST API integration
- Relational database with migrations
- Validation and error handling
- Audit-friendly activity records
- Deployment-ready configuration

6. Functional / Professional Requirements

- Define the target user/stakeholder, scope boundaries and success metrics before execution.
- Use a repeatable workflow with traceable decisions and clearly organized artifacts.
- Handle realistic edge cases, invalid inputs, exceptions or uncertainty relevant to the domain.
- Include a validation/review step and retain evidence of the result.
- Document limitations and recommend realistic next improvements.

7. Technical / Method Requirements

- Recommended stack: React, REST API, PostgreSQL, role-based access, Docker
- Use current stable tooling that is accessible to the intern; reasonable alternatives are allowed when documented.
- Apply version control or a dated evidence folder appropriate to the domain.
- Follow responsible-use, security, privacy and data-integrity practices relevant to the project.

8. Suggested Technology Stack

React, REST API, PostgreSQL, role-based access, Docker

9. Step-by-Step Implementation Plan

1. Independently refine requirements after reviewing edge cases, constraints and stakeholder priorities.
2. Build the main solution with a clear separation between inputs, processing, outputs and reporting.
3. Add validation, error handling, quality controls and measurable performance/accuracy/usability checks.
4. Run a structured review against the acceptance criteria and correct the highest-impact gaps.
5. Prepare a concise professional demonstration supported by evidence and documentation.

10. Week-by-Week Plan

ARROWSTACK
LEARN. BUILD. GROW.

Week 1	Research, stakeholder/problem framing, requirements, dataset/assets, environment setup and initial design.
Week 2	Core implementation, analysis, content creation, research execution or prototype build.
Week 3	Advanced requirements, testing/validation, optimization, refinement and mentor review.
Week 4	Final QA/validation, documentation, presentation/demo, packaging and submission.

11. Deliverables

- Project source files or working analysis/design artifacts
- Requirements and implementation/research documentation
- Test/validation evidence
- README or user/technical guide
- Final presentation or demo
- Version-controlled project repository or organized submission folder

12. Expected Final Output

A complete, organized and presentation-ready project package that demonstrates the required workflow, includes evidence of quality/validation, and can be understood by a mentor without relying on undocumented steps.

13. Acceptance Criteria

- All stated core requirements are completed and mapped to evidence.
- The solution/workflow is reproducible by a mentor using the submitted instructions and files.
- Quality checks/tests/validation are documented and major known issues are disclosed.
- The final output is professionally organized, named consistently and ready for demonstration.
- The intern can explain key decisions, trade-offs, limitations and future improvements.

14. Evaluation Rubric — 100 Points

Criterion	Points
Requirements & domain understanding	15
Technical / professional execution	25
Quality, accuracy & validation	20
Problem-solving & independence	15
Documentation & reproducibility	10
Presentation & communication	10
Creativity / improvement thinking	5

15. Portfolio Value

Present the project on a resume as a measurable internship project; publish appropriate source/artifacts on GitHub or a portfolio; share a concise outcome-focused LinkedIn post; and use the project as an interview story covering problem, action, evidence and result.

16. Viva / Interview Questions

- What business or user problem does the project solve, and how did you define success?
- Which design, technical, analytical or research decision had the biggest impact on the result?
- How did you validate the quality or correctness of your work?
- What edge cases, limitations or risks did you identify?
- What would you change if this solution had to support 10× the users/data/workload?
- How would you explain the project to a non-technical stakeholder?
- Which part of the project required the most independent problem-solving?

17. Extension / Advanced Challenge

ARROWSTACK
LEARN. BUILD. GROW.

- Add monitoring/measurement or a stronger evaluation framework and compare results before vs. after.
- Introduce an additional user/stakeholder role or more realistic edge cases.
- Automate one repetitive step and document the productivity or quality benefit.
- Create a production/research-readiness roadmap covering security, scalability, governance or maintainability.

ARROWSTACK
LEARN. BUILD. GROW.

Project 3: Subscription Analytics SaaS MVP

Project Type: Capstone / Portfolio Task

1. Real-World Scenario

Create a SaaS-style application that manages plans, customer accounts, usage records, billing events and business analytics.

2. Project Objective

Deliver a working, reviewable solution for the scenario using professional full stack development practices. The objective is not only to produce an output, but to show how you moved from problem definition to a defensible final result.

3. Business Use Case

A real organization could use this type of workflow to improve service quality, operational visibility, productivity, decision-making or customer/user experience. The intern should identify the primary stakeholder, the decision enabled by the output and the measurable benefit.

4. Problem Statement

Design and complete the full stack development project so that a stakeholder can reliably use or evaluate the final result. Define inputs, outputs, constraints, quality measures and the conditions under which the solution should be considered successful.

5. Required Features / Activities

- Role-based access
- REST API integration
- Relational database with migrations
- Validation and error handling
- Audit-friendly activity records
- Deployment-ready configuration

6. Functional / Professional Requirements

- Define the target user/stakeholder, scope boundaries and success metrics before execution.
- Use a repeatable workflow with traceable decisions and clearly organized artifacts.
- Handle realistic edge cases, invalid inputs, exceptions or uncertainty relevant to the domain.
- Include a validation/review step and retain evidence of the result.
- Document limitations and recommend realistic next improvements.

7. Technical / Method Requirements

- Recommended stack: Next.js, Node.js, PostgreSQL, charts, Stripe-like simulated billing
- Use current stable tooling that is accessible to the intern; reasonable alternatives are allowed when documented.
- Apply version control or a dated evidence folder appropriate to the domain.
- Follow responsible-use, security, privacy and data-integrity practices relevant to the project.

8. Suggested Technology Stack

Next.js, Node.js, PostgreSQL, charts, Stripe-like simulated billing

9. Step-by-Step Implementation Plan

1. Frame the capstone as a portfolio project with a defined business owner, user group and measurable outcome.
2. Design the end-to-end solution, including architecture/design/research method, data or assets and risk controls.
3. Implement the highest-value features and perform testing, validation, optimization or peer review.
4. Package the work as a professional handoff with reproducibility, documentation and presentation materials.
5. Reflect on trade-offs, limitations and realistic next steps for a production or research environment.

10. Week-by-Week Plan

ARROWSTACK
LEARN. BUILD. GROW.

Week 1	Research, stakeholder/problem framing, requirements, dataset/assets, environment setup and initial design.
Week 2	Core implementation, analysis, content creation, research execution or prototype build.
Week 3	Advanced requirements, testing/validation, optimization, refinement and mentor review.
Week 4	Final QA/validation, documentation, presentation/demo, packaging and submission.

11. Deliverables

- Project source files or working analysis/design artifacts
- Requirements and implementation/research documentation
- Test/validation evidence
- README or user/technical guide
- Final presentation or demo
- Version-controlled project repository or organized submission folder

12. Expected Final Output

A complete, organized and presentation-ready project package that demonstrates the required workflow, includes evidence of quality/validation, and can be understood by a mentor without relying on undocumented steps.

13. Acceptance Criteria

- All stated core requirements are completed and mapped to evidence.
- The solution/workflow is reproducible by a mentor using the submitted instructions and files.
- Quality checks/tests/validation are documented and major known issues are disclosed.
- The final output is professionally organized, named consistently and ready for demonstration.
- The intern can explain key decisions, trade-offs, limitations and future improvements.

14. Evaluation Rubric — 100 Points

Criterion	Points
Requirements & domain understanding	15
Technical / professional execution	25
Quality, accuracy & validation	20
Problem-solving & independence	15
Documentation & reproducibility	10
Presentation & communication	10
Creativity / improvement thinking	5

15. Portfolio Value

Present the project on a resume as a measurable internship project; publish appropriate source/artifacts on GitHub or a portfolio; share a concise outcome-focused LinkedIn post; and use the project as an interview story covering problem, action, evidence and result.

16. Viva / Interview Questions

- What business or user problem does the project solve, and how did you define success?
- Which design, technical, analytical or research decision had the biggest impact on the result?
- How did you validate the quality or correctness of your work?
- What edge cases, limitations or risks did you identify?
- What would you change if this solution had to support 10× the users/data/workload?
- How would you explain the project to a non-technical stakeholder?
- Which part of the project required the most independent problem-solving?

17. Extension / Advanced Challenge

ARROWSTACK
LEARN. BUILD. GROW.

- Add monitoring/measurement or a stronger evaluation framework and compare results before vs. after.
- Introduce an additional user/stakeholder role or more realistic edge cases.
- Automate one repetitive step and document the productivity or quality benefit.
- Create a production/research-readiness roadmap covering security, scalability, governance or maintainability.

ARROWSTACK
LEARN. BUILD. GROW.